

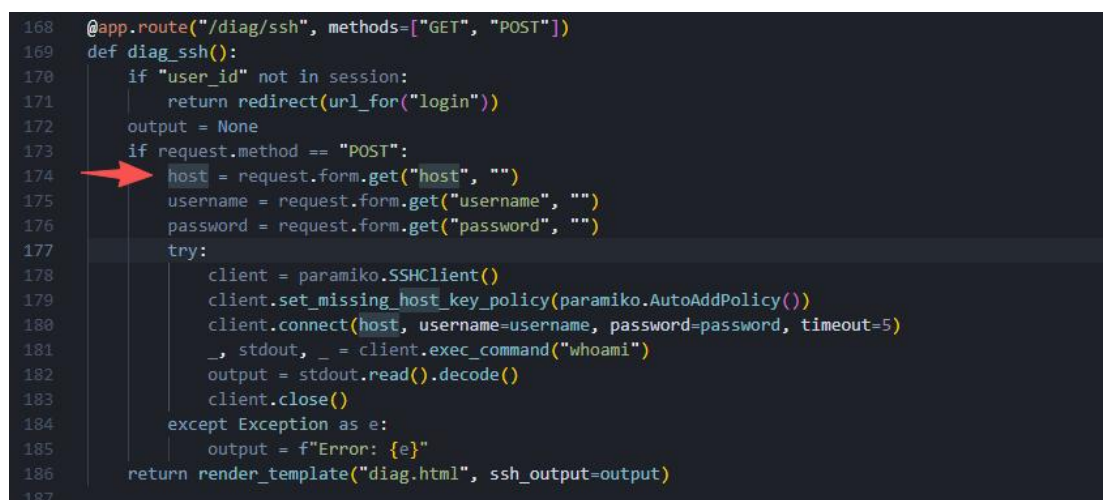
Semgrep 代码分析实验报告

李琪轩 25140931

一、源码问题分析

在 code-repo/app.py 中，分析一个具体的问题。本次实验分析的是一个命令注入漏洞。

首先分析源码 source:



```
168 @app.route("/diag/ssh", methods=["GET", "POST"])
169 def diag_ssh():
170     if "user_id" not in session:
171         return redirect(url_for("login"))
172     output = None
173     if request.method == "POST":
174         host = request.form.get("host", "")
175         username = request.form.get("username", "")
176         password = request.form.get("password", "")
177         try:
178             client = paramiko.SSHClient()
179             client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
180             client.connect(host, username=username, password=password, timeout=5)
181             _, stdout, _ = client.exec_command("whoami")
182             output = stdout.read().decode()
183             client.close()
184         except Exception as e:
185             output = f"Error: {e}"
186     return render_template("diag.html", ssh_output=output)
187
```

数据来自用户前端表单输入，完全不可信。

分析 path:

用户输入的 host 变量未经过任何过滤、净化、校验，直接拼接到系统命令字符串中。

总结 sink:

```

@app.route("/diag/ping", methods=["GET", "POST"])
def diag_ping():
    if "user_id" not in session:
        return redirect(url_for("login"))
    output = None
    if request.method == "POST":
        host = request.form.get("host", "")
        # B602: subprocess with shell=True and unsanitized user input - command injection
        output = subprocess.check_output("ping -c 1 " + host, shell=True,
                                         stderr=subprocess.STDOUT).decode()
    return render_template("diag.html", output=output)

```

脏数据流入 subprocess 执行函数，且开启 shell=True，触发命令注入。

二、基于 semgrep 的规则编写

规则文件如下：

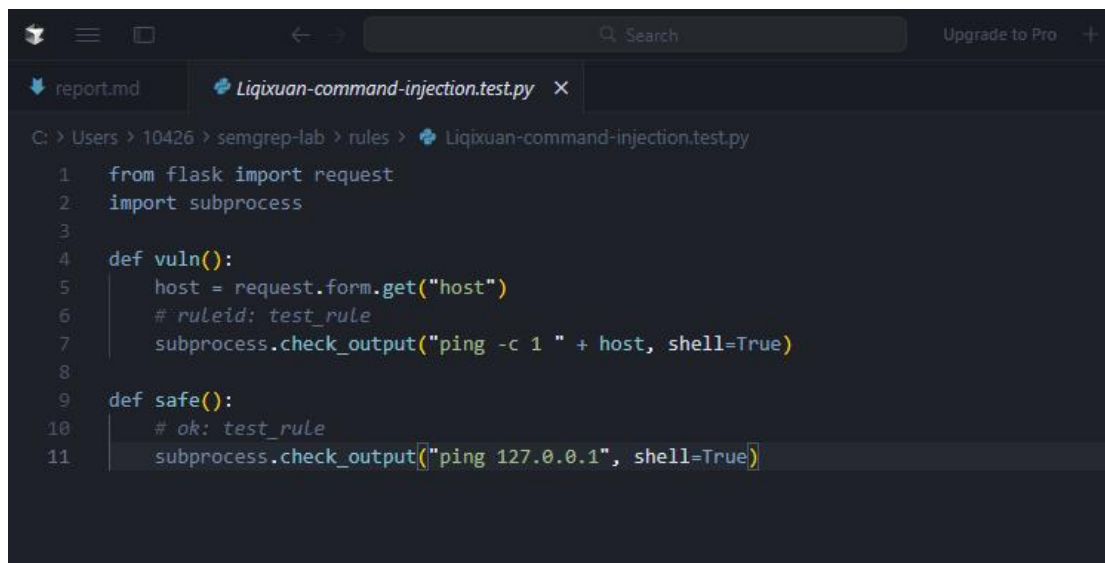
```

C:\> Users > 10426 > semgrep-lab > rules > ! Liqixuan-command-injection.yaml

1  rules:
2  - id: test_rule
3    message: command injection
4    severity: ERROR
5    languages: [python]
6    mode: taint
7    pattern-sources:
8      - pattern: request.form.get(...)
9      - pattern: request.args.get(...)
10   pattern-sinks:
11     - pattern: subprocess.$FUNC(..., shell=True, ...)
12       metavariable-regex:
13         metavariable: $FUNC
14       regex: "^(check_output|run|call|Popen)$"

```

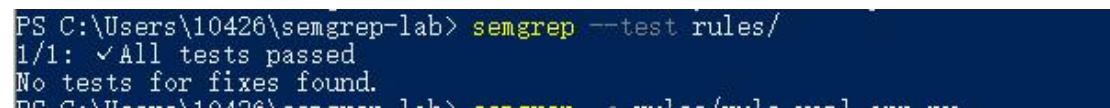
编写测试文件：



The screenshot shows a code editor with a dark theme. The top bar includes a search icon, a search input field, and an 'Upgrade to Pro' button. The editor has two tabs: 'report.md' and 'Liqixuan-command-injection.test.py'. The active tab displays a Python script with the following content:

```
C: > Users > 10426 > semgrep-lab > rules > Liqixuan-command-injection.test.py
1  from flask import request
2  import subprocess
3
4  def vuln():
5      host = request.form.get("host")
6      # ruleid: test_rule
7      subprocess.check_output("ping -c 1 " + host, shell=True)
8
9  def safe():
10     # ok: test_rule
11     subprocess.check_output("ping 127.0.0.1", shell=True)
```

运行测试:



The screenshot shows a terminal window with the following output:

```
PS C:\Users\10426\semgrep-lab> semgrep --test rules/
1/1: ✓All tests passed
No tests for fixes found.
PS C:\Users\10426\semgrep-lab> semgrep --test rules/rules-test-sem-se
```

三、执行漏洞扫描

执行结果如下

```
PS C:\Users\10426\semgrep-lab> semgrep -c rules/Liqixuan-command-injection.yaml target/code-repo
```

○○○
Semgrep CLI

[Loading rules...
Scanning 13 files (only git-tracked) with 1 Code rule:

CODE RULES
Scanning 3 files.

SUPPLY CHAIN RULES

No rules to run.

PROGRESS

0:00:00

2 Code Findings

```
target\code-repo\app.py
[1] rules.test_rule
[1][Blocking][1]
command injection

134 | result = subprocess.call(["sh", "-c", "cat " + filepath], shell=True)
    | -----
197 | output = subprocess.check_output("ping -c 1 " + host, shell=True,
198 |                                stderr=subprocess.STDOUT).decode()
```

Scan Summary

Scan completed successfully.
• Findings: 2 (2 blocking)
• Rules run: 1
• Targets scanned: 3
• Parsed lines: ~100.0%
• No ignore information available
Ran 1 rule on 3 files: 2 findings.